

CLOUD NATIVE ARCHITECTURE INFRASTRUCTURE

Proxmox 기반 하이브리드클라우드 인프라 구축 및 고가용성 설계

2026.05.28 KOSA 클라우드 엔지니어링 최종 발표

발표 목차

01

인트로

표지, 팀원 소개, 배경 시나리오

02

아키텍처

하이브리드 클라우드 전체 구성

03

인프라

네트워크, AWS, DB

04

Devops

Gitea, Actions, Harbor, Argocd

05

Observability

Prometheus, Grafana, Loki 통합 관제

06

보안

Wazuh 기반 취약점 점검 및 감사

07

한계

한계 및 개선

프로젝트 수행 팀원 소개



손정원

인프라 총괄

아키텍처 설계
AWS 및 CI/CD 구축



유진주

클러스터 구축

온프레미스 설계 및 구축
k8s 클러스터 구축



정혜윤

네트워크 설계

네트워크 설계
온프레미스/AWS VPC 구축



이해인

관제 시스템

Observability 시스템 구축
(Grafana, Prometheus, Loki)



김소연

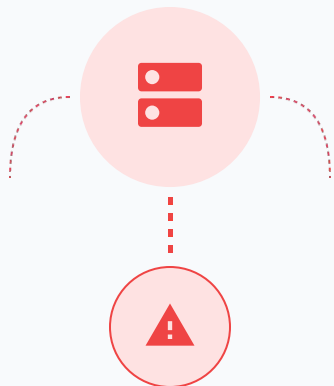
보안 시스템

보안 시스템 구축 및 점검
(Wazuh)

인트로

단일 서버의 한계 극복 및 하이브리드 인프라 전환

AS-IS



단일 서버 운영 한계

트래픽 급증으로 인한 성능 저하 및 단일 장애 지점 (SPOF) 노출 위험

TO-BE



온프레미스 K8s 클러스터 구축

4대 물리 노드 기반 가상화 및 컨테이너 자원 분리



분산 스토리지 Ceph 도입

6대 서버 구성으로 데이터 고가용성 및 안정성 확보

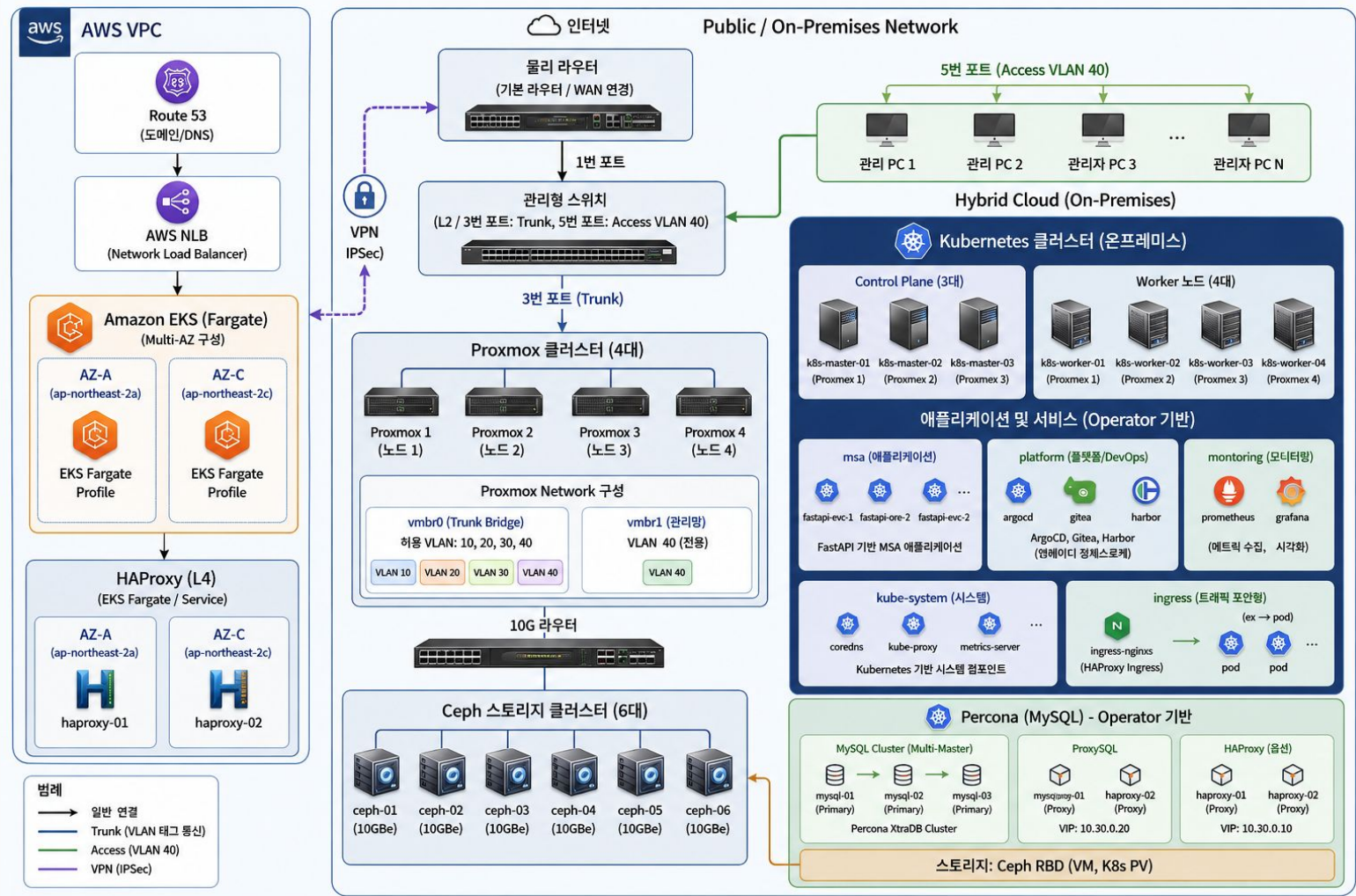


하이브리드 클라우드 연계

AWS를 활용한 외부 진입점 분리 및 재해 복구(DR) 체계

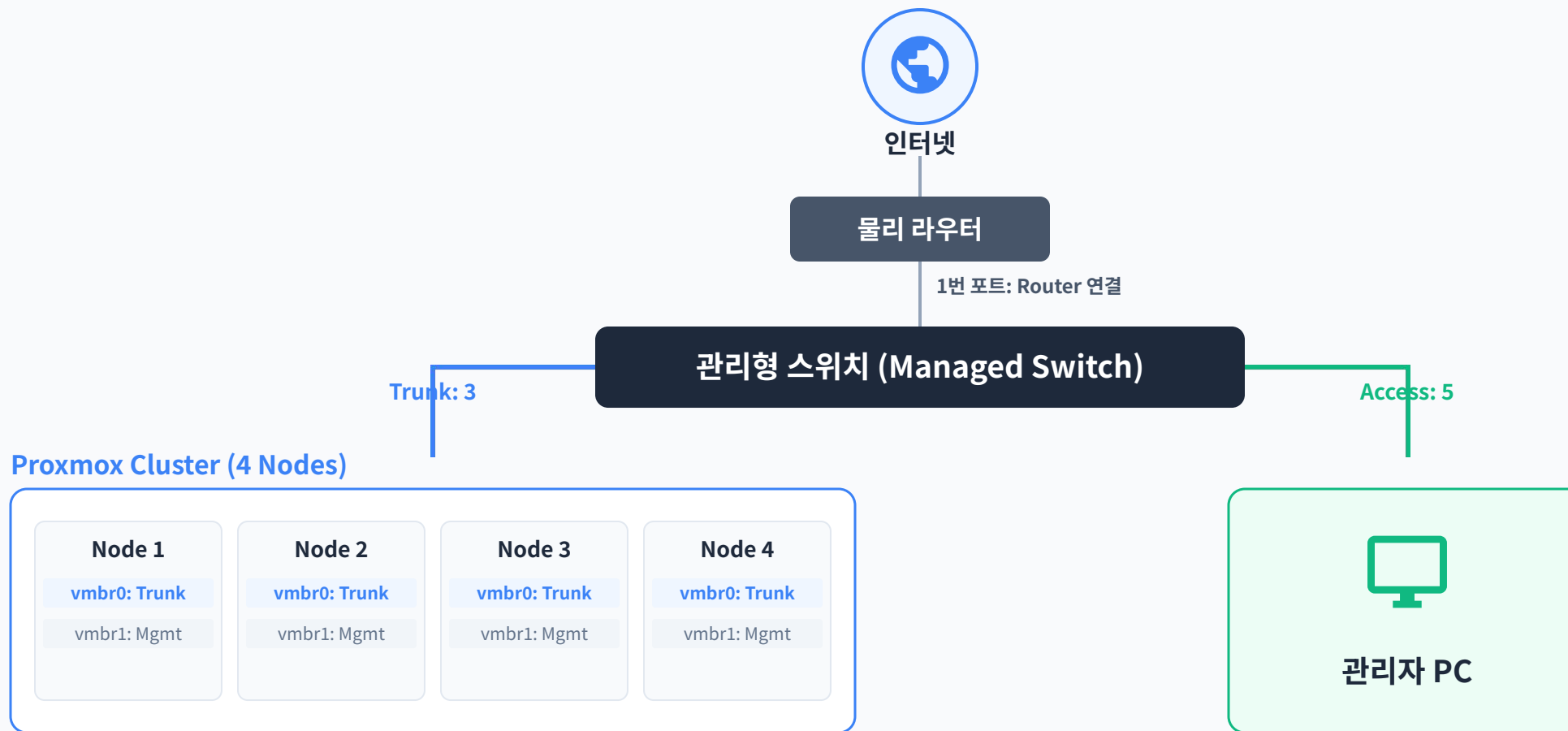
결과: 제한된 예산 내에서 **안정성·확장성·운영 효율성** 극대화

아키텍처



인프라

물리 네트워크 연결 구조



Trunk 포트와 Access 포트를 분리하여 서비스망과 관리 접근망을 구분 구성

1G / 10G 네트워크 역할 분리

1G Network

관리 및 외부 접근

포함 요소

- 인터넷 / 물리 라우터, 관리형 스위치
- 관리자 PC, Proxmox Mgmt

주요 트래픽

- 관리자 접속 및 장비 관리
- 외부망 연결 트래픽



10G Network

스토리지 전용 고속망

포함 요소

- Proxmox Cluster, Ceph Storage
- VM Disk, K8s PV

주요 트래픽

- Ceph RBD 데이터 전송 및 디스크 I/O
- K8s PV 스토리지 트래픽



관리·외부 접근망과 Ceph 스토리지망을 분리하여 성능과 안정성 확보

방화벽 Vlan 정책 구성

10 공용망

20 DMZ

30 내부망

40 관리망

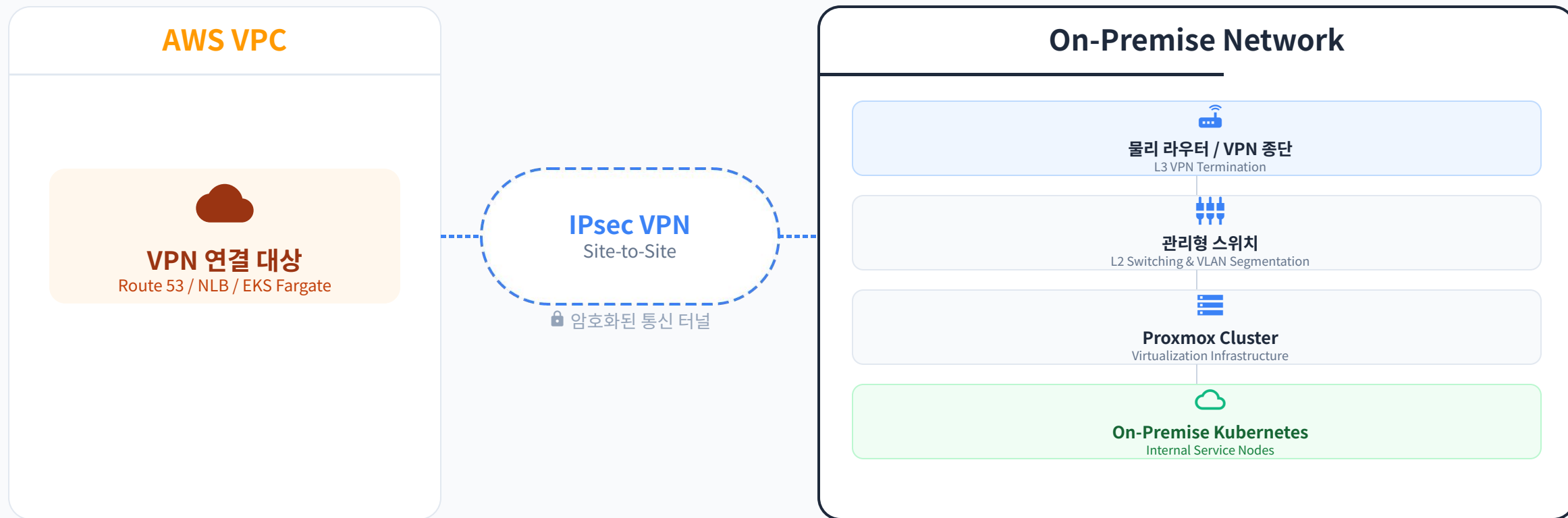
출발지	도착지	아웃바운드	인바운드	설명
VLAN10	VLAN20	●	●	-
VLAN10	VLAN30	✗	●	공용망 -> 내부망 차단
VLAN 10	VLAN 40	✗	●	공용망 -> 관리망 차단
VLAN20	VLAN 40	●	✗	DMZ -> 관리망 차단
VLAN20	VLAN 40	●	✗	관리망 -> DMZ 차단
VLAN30	VLAN 40	●	✗	관리망 -> 내부망 차단
VLAN 30	VLAN 40	●	✗	관리망 -> 내부망 SSH 허용

Vlan 역할 구성



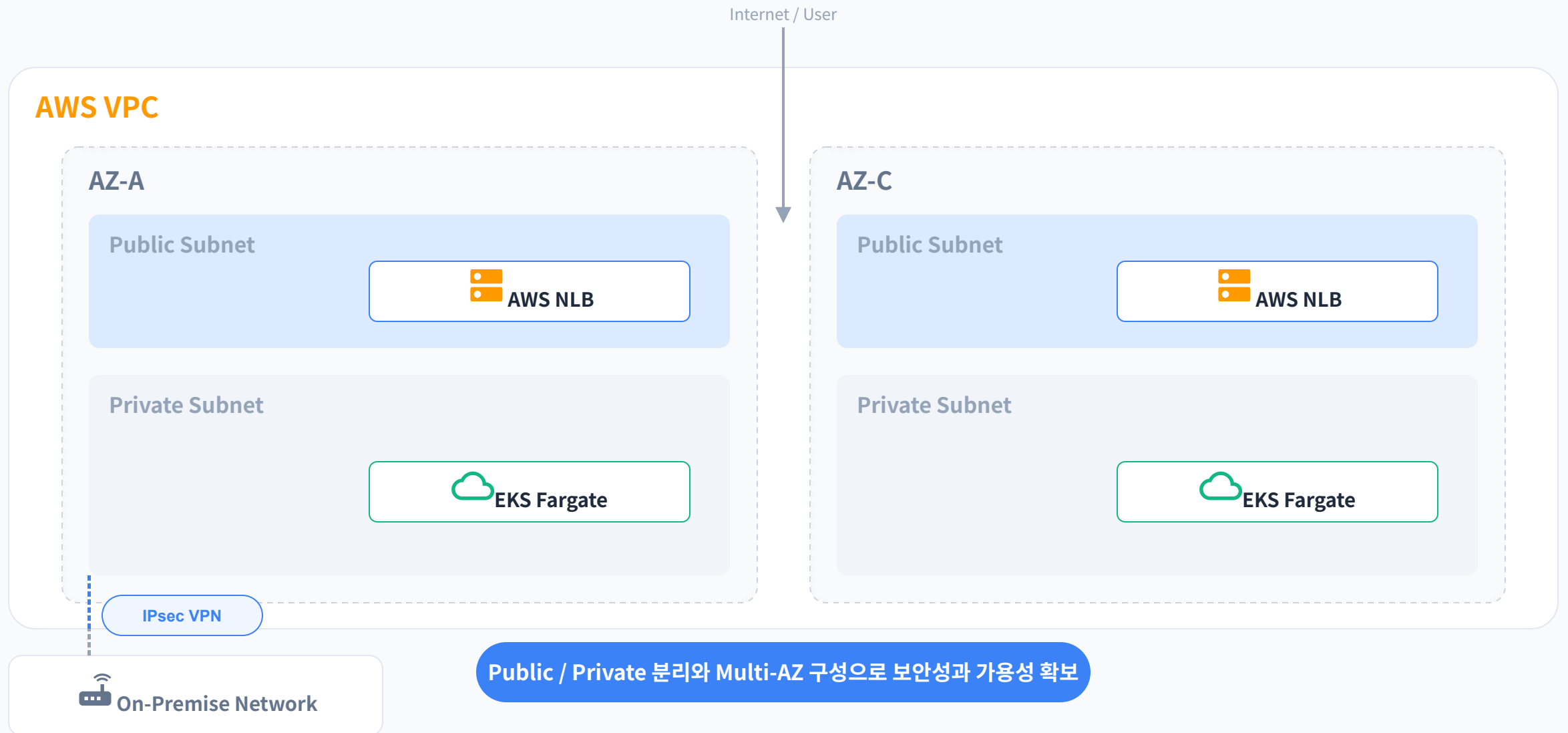
VLAN별 역할을 분리하여 서비스 진입(LB), 메인 서비스망(Prod), 관리 접근망(Mgmt)을 구분 구성

IPsec VPN 연동 구조

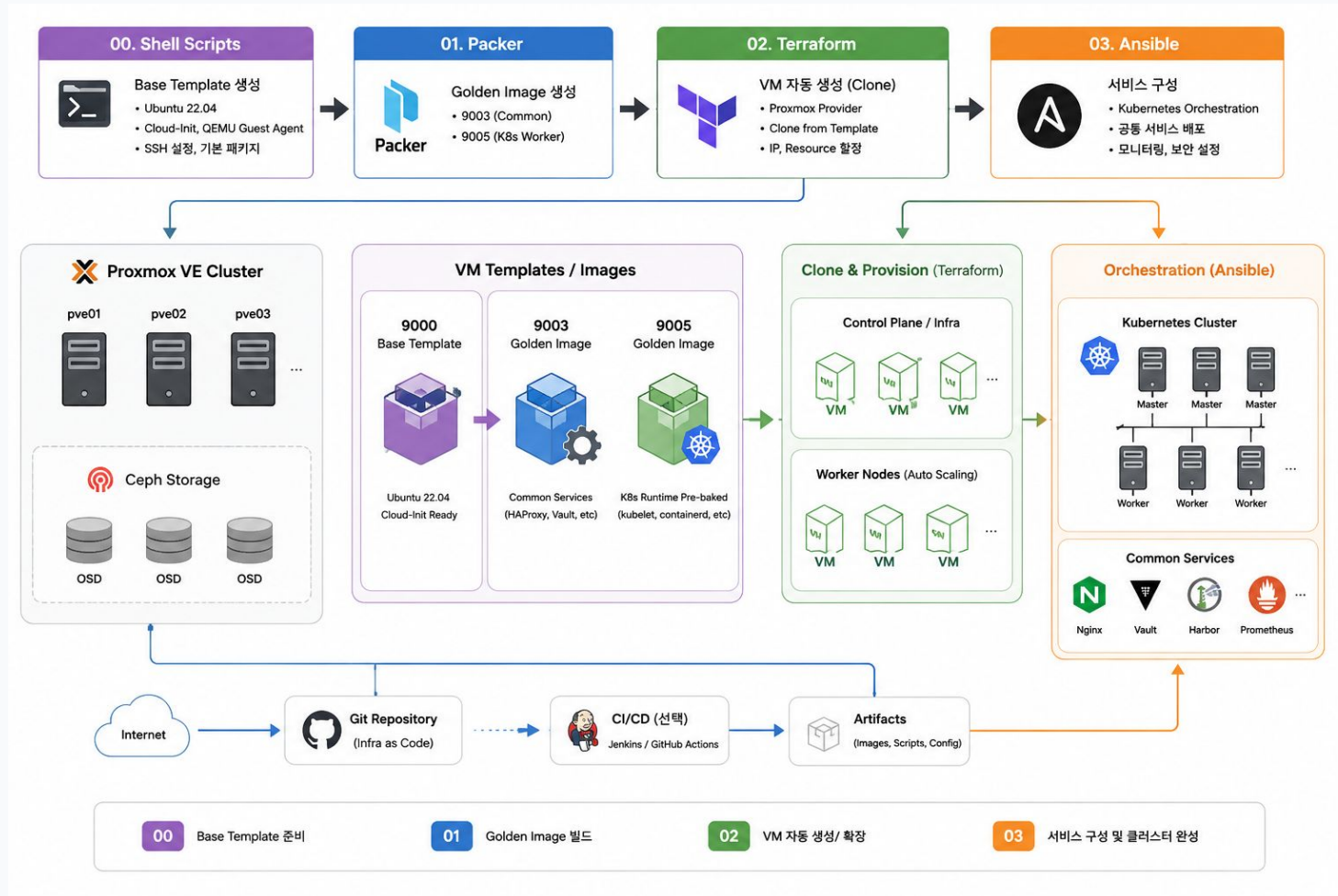


AWS VPC와 온프레미스 네트워크를 IPsec VPN으로 연결

AWS VPC 네트워크

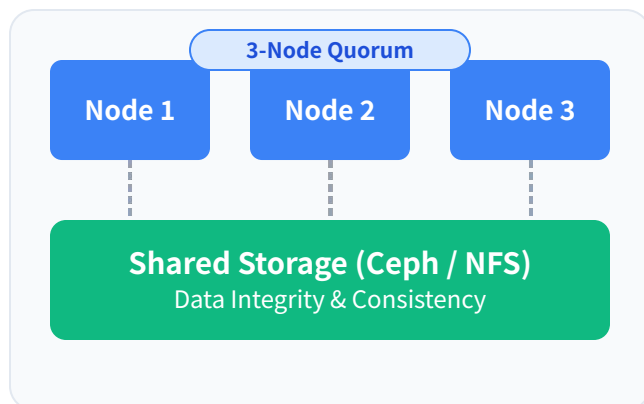


IaC 인프라 구성



Proxmox HA 클러스터 구성

고가용성(HA) 아키텍처



- 3-Node Quorum 구성으로 스플릿 브레인 방지
- 공유 스토리지(Ceph/NFS) 기반 데이터 정합성
- Corosync 기반 실시간 노드 상태 모니터링



01. 자동 장애 감지 (Failover)

클러스터 멤버 노드 중 하나에서 하드웨어 또는 네트워크 장애 발생 시, HA 매니저가 즉각적으로 이상 상태 감지



02. VM 자동 마이그레이션

장애 노드의 VM 리소스를 가용 노드로 즉시 재할당하여 서비스 중단 최소화 및 재개

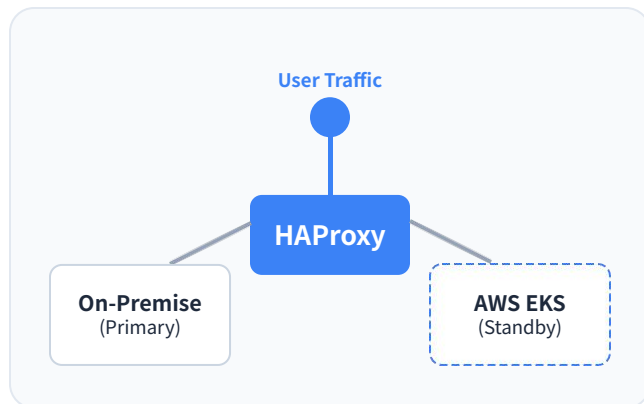


03. 헬스 체크 및 셀프 힐링

Corosync 기반 실시간 쿼럼 상태 모니터링 및 비정상 서비스 발생 시 자동 복구 프로세스 가동

Hybrid-Cloud HA 구성

HAProxy 로드밸런싱 아키텍처



- HAProxy 기반 하이브리드 트래픽 제어
- On-premise K8s 우선 순위 배치
- 장애 시 EKS(Cloud) 자동 페일오버



01. 하이브리드 백엔드 구성

HAProxy 뒷단에 온프레미스 K8s와 AWS EKS를 동시 배치하여 단일 엔드포인트 서비스 제공



02. 상태 확인 및 가용성 감시

HAProxy의 Health Check 기능을 통해 온프레미스 노드 가용 상태 실시간 모니터링

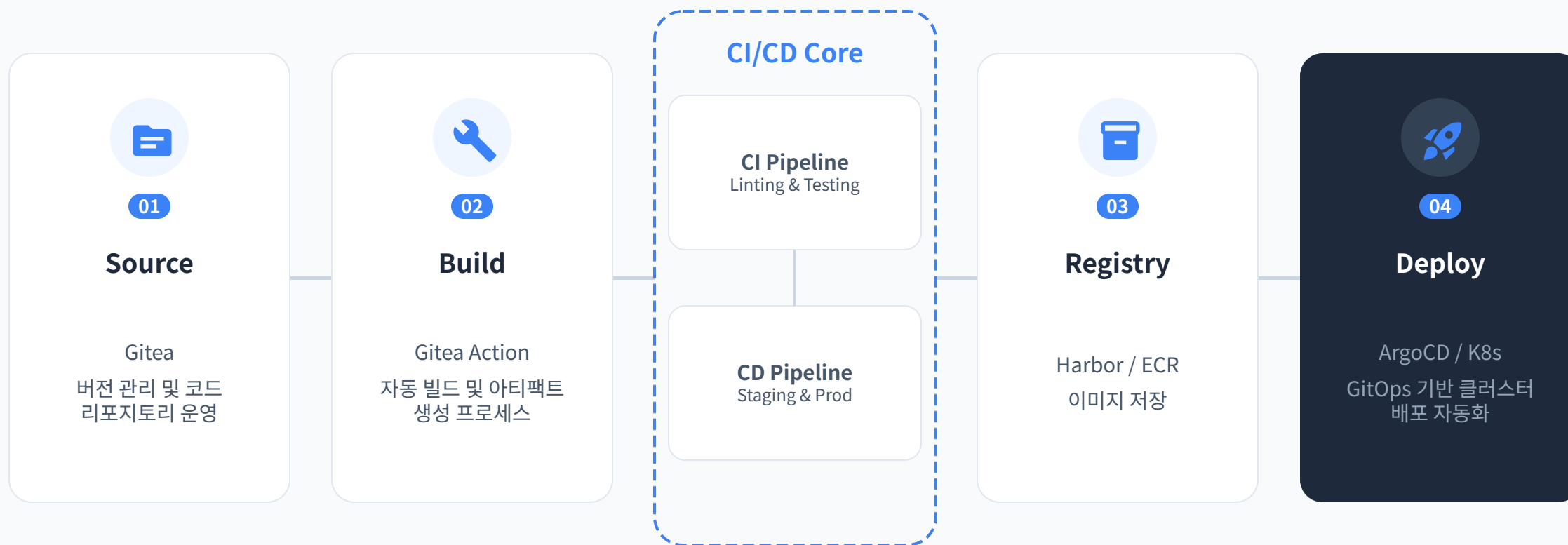


03. 무중단 페일오버 (Failover)

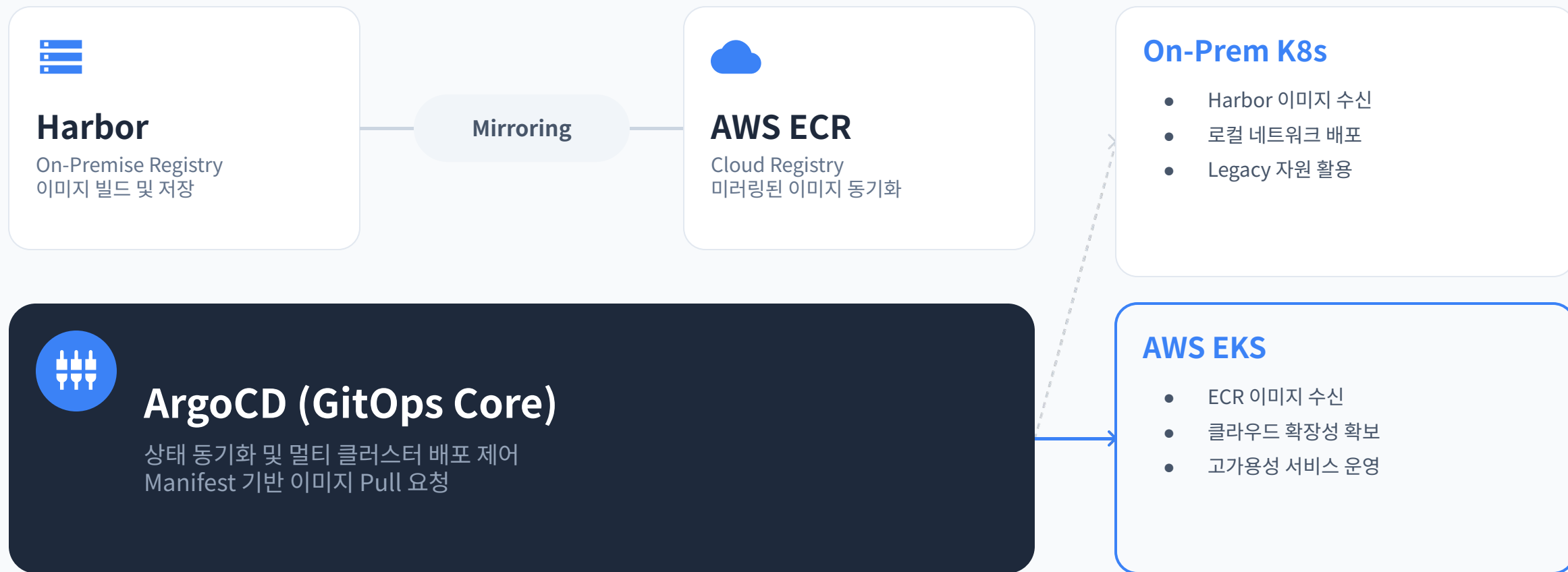
온프레미스 장애 시 트래픽 즉시 EKS 전환, 서비스 연속성 보장 및 가동 중단 방지

Devops

인프라 DevOps CI / CD Pipeline



인프라 DevOps ArgoCD 멀티 클러스터 배포



Observability

통합 관제 시스템 도입 배경 및 과제

분산 인프라 가시성 부재 해결

Proxmox 노드 4대와 K8s 내부망에 분산된 마이크로서비스의 개별 헬스 체크 및 추적 관리가 불가능했던 물리적 환경 극복

수동 장애 대응 체계의 전면적 혁신

기존 SSH 접속 및 **grep 기반 사후 분석** 위주의 비효율적인 운영 방식을 개선하고 자동화된 대응 체계 구축

보안 및 데이터 규제(Compliance) 완벽 대응

핵심 비즈니스 데이터의 안전한 관리를 위한 접근 제어(auth) 및 시스템 이벤트 로그의 중앙 집중형 보관 정책 수립

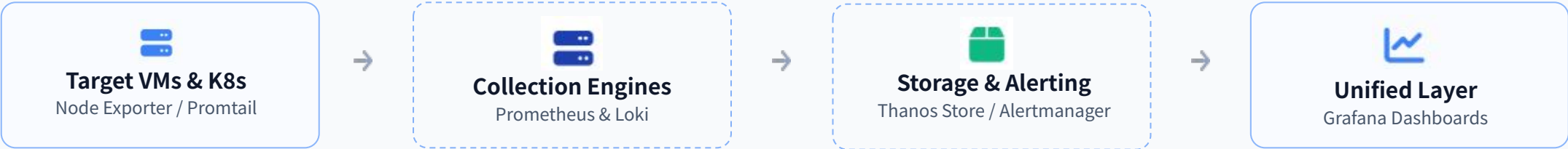
관제 고도화 달성 성과

하이브리드 확장망 전반의 메트릭과 로그를 **단일 가시성 저장소**로 통합하여 시스템 신뢰성 확보

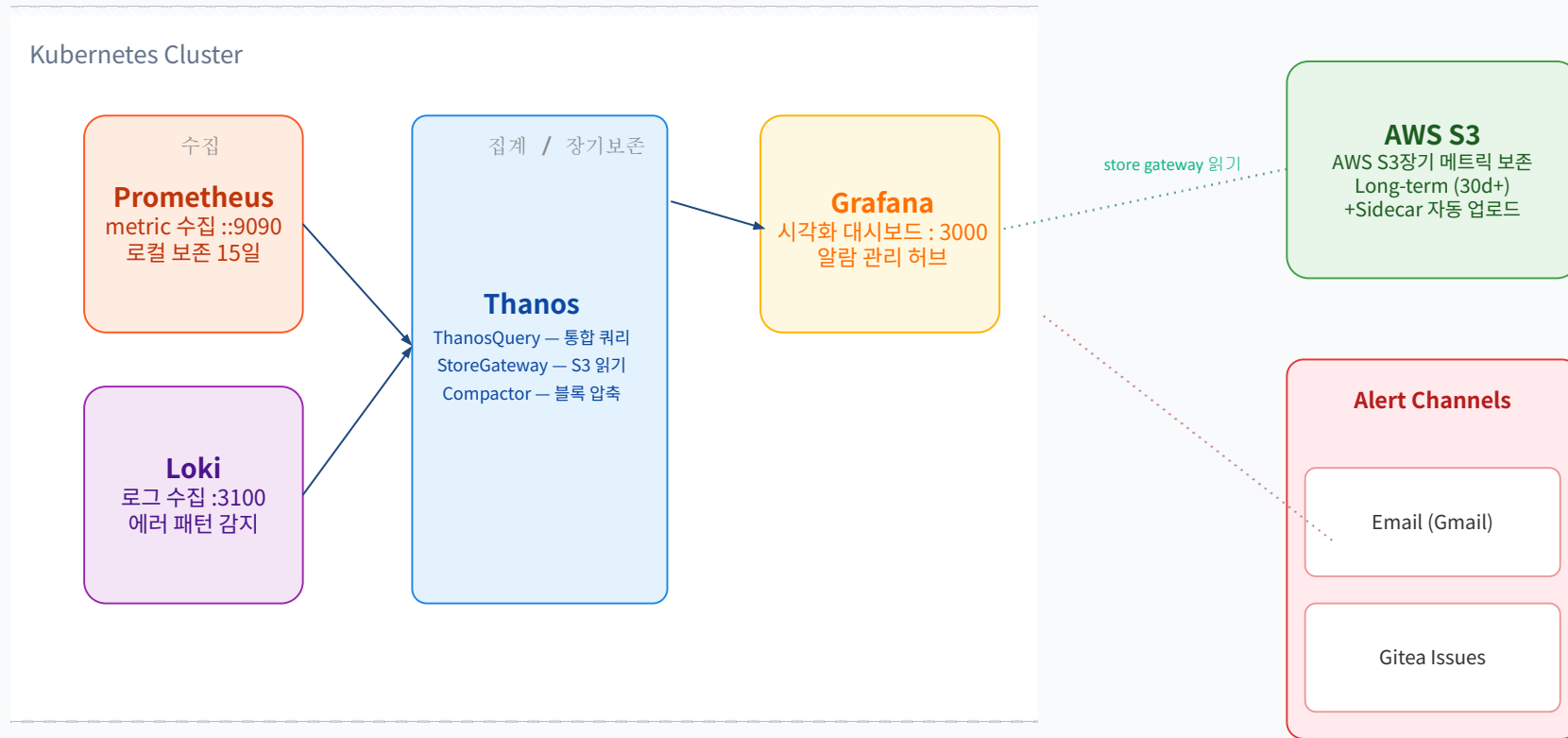
장애 대응 핵심 지표인 **MTTR**을 기존 분 단위에서 **초 단위**로 획기적으로 단축하는 관제 자동화 구현

메트릭 및 로그 통합 수집 파이프라인

-  **시계열 메트릭 파이프라인 (Prometheus & Node Exporter)** 개별 VM 운영체제 레이어에 Node Exporter 배치 및 쿠버네티스 Kube-State-Metrics 연동. 15초 주기 풀링(Pull) 방식으로 중앙 Prometheus 타겟에 시스템 자원 현황 적재
-  **스토리지 장기 보존 및 HA 확보 (Thanos 연동)** Prometheus의 단일 장애점(SPOF) 해소 및 스토리지 휘발성 보완. Thanos Query·Store 아키텍처 도입을 통한 영구 스토리지 캐싱 환경 구축
-  **중앙 집중형 로깅 파이프라인 (Loki & Promtail)** 분산 VM 인프라 로그 수집용 경량 에이전트 Promtail 가동. 파일 인덱스를 최소화한 Loki 아키텍처 기반의 실시간 스트리밍(Push) 체계 구현



통합 모니터링 시스템 아키텍처



*Prometheus 로컬 15일 → Thanos Compactor 압축/병합 → AWS S3 30일+ 무제한

*Prometheus의 15일 한계를 극복. AWS S3 연동으로 30일 이상의 데이터를 무제한 보관

쿠버네티스 클러스터 자원 현황 및 위계 관제

7

Total Active Nodes

106

Total Running Pods



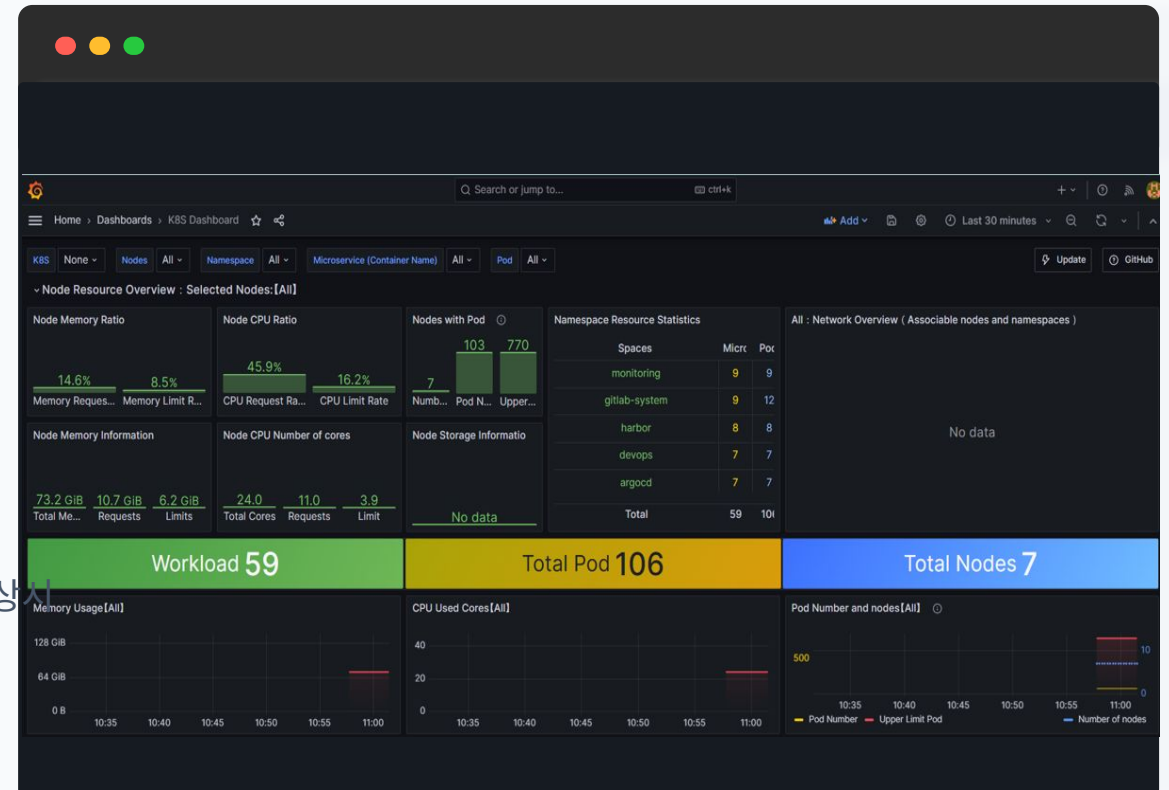
클러스터 상태 실시간 검증

총 7개의 마스터 및 워커 노드가 클러스터 맵에 정상 바인딩되어 안정적으로 동작 중임을 확인

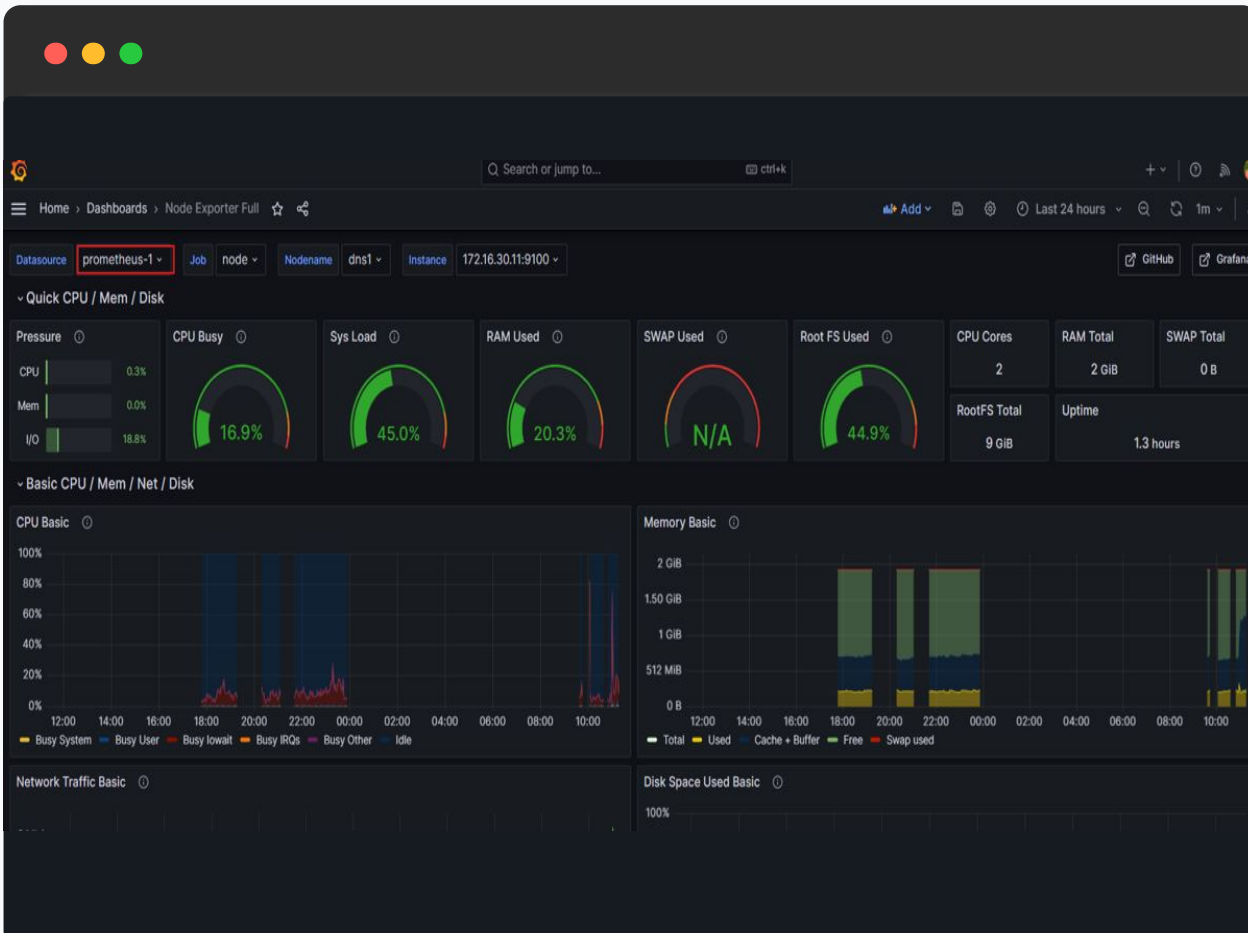


고밀도 분산 컨테이너 관리

MSA, 플랫폼 등 총 106개 Pod의 위계 모델이 정상 스케줄링 상태임을 상시 모니터링



Node Exporter 기반 노드별 세부 메트릭 분석



2.10 GiB

Current Memory Usage

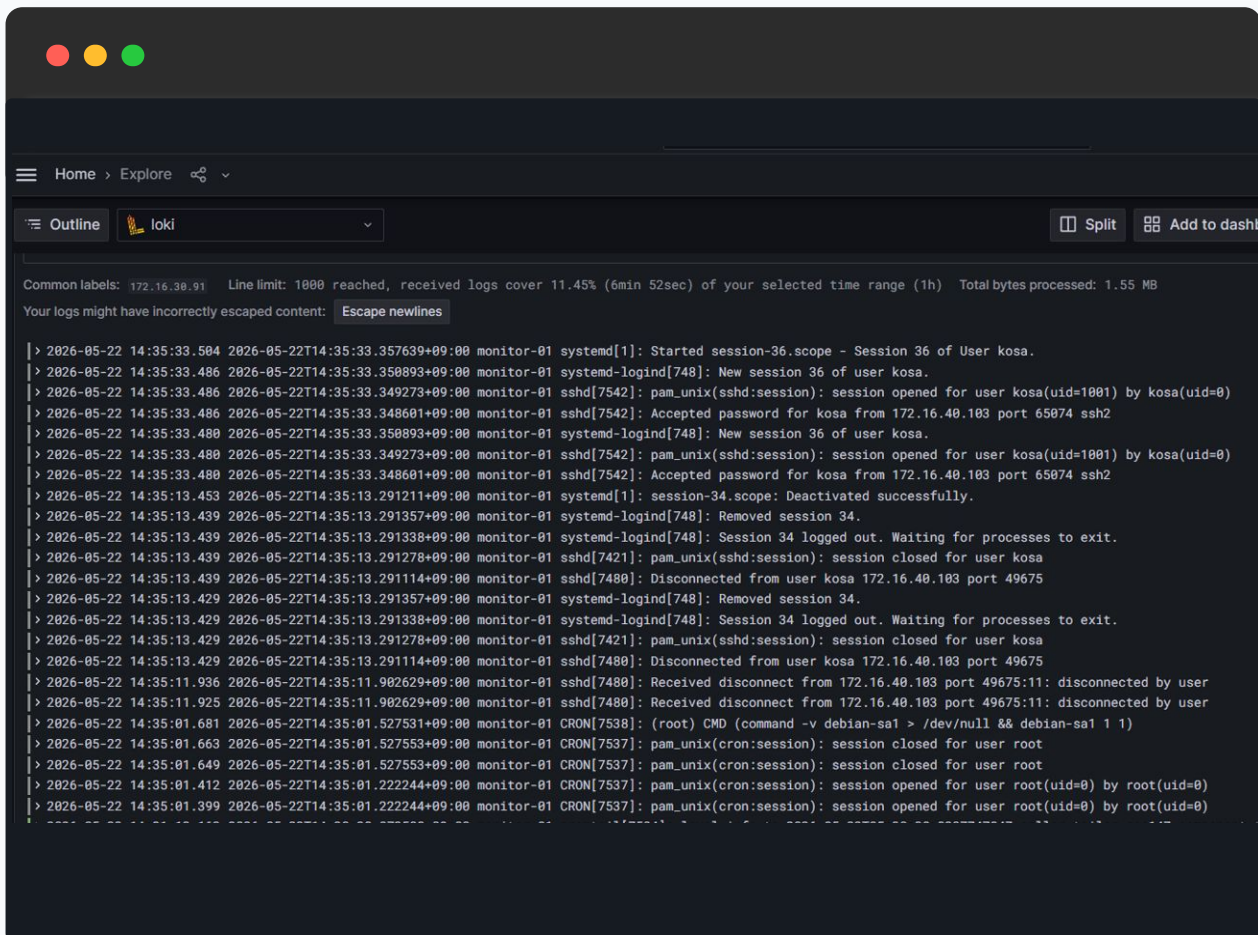
44.9%

Root FS Disk Storage

🔧 **멀티코어 CPU 사용량 파싱** 각 인프라의 CPU 사용률 변동 추이를 시각화하여, 클라우드 버스팅 트리거 임계치(70%) 도달 여부 선제 모니터링

🏠 **자원 격리 상태 실시간 매핑** 메모리의 유효 캐시/버퍼 용량을 정밀 추적하여 비정상적인 메모리 누수(Memory Leak) 현상 사전 차단

Loki 레이블 브라우저를 통한 시스템 내부 보안 감사 로그 정밀 스트리밍



원격 접속(sshd) 보안 추적 완비

내부 서버 접근 세션의 개폐 내역 실시간 파싱, 외부 침입 및 불법 root 접근 시도 철저히 모니터링



정밀 타임스탬프 기반 사후 포렌식

밀리초(ms) 단위 고해상도 로그 타임라인 제공, 인프라 장애 시점의 선후 인과 관계를 명확히 추적



통합 엔드포인트 단일 뷰 가시성

개별 노드 터미널 접속 없이 중앙 대시보드에서 전 인프라 로그를 실시간 스트리밍 형태로 통합 관제

주요 모니터링 알람 규칙

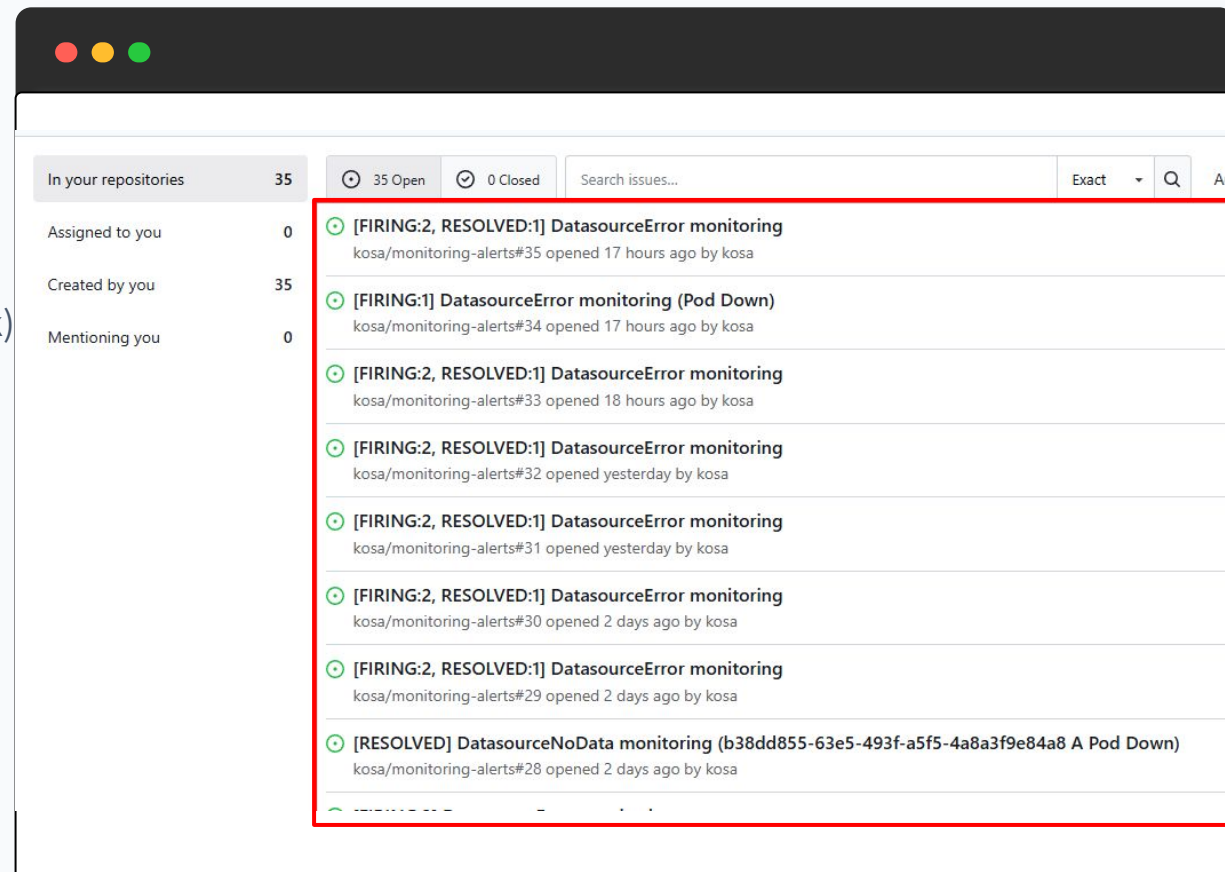
알람명 (Alert Rule)	임계치 및 심각도 (Condition & Severity)	판정 시간
High CPU Usage	사용률 > 80% 지속 Warning	2분
High Memory Usage	사용률 > 80% 지속 Warning	2분
Pod CrashLoopBackOff	재시작 횟수 빈번 Critical	1분
Node NotReady	K8s 노드 상태 비정상 Critical	1분

Alertmanager & Gitea 연동 지능형 알람 자동화

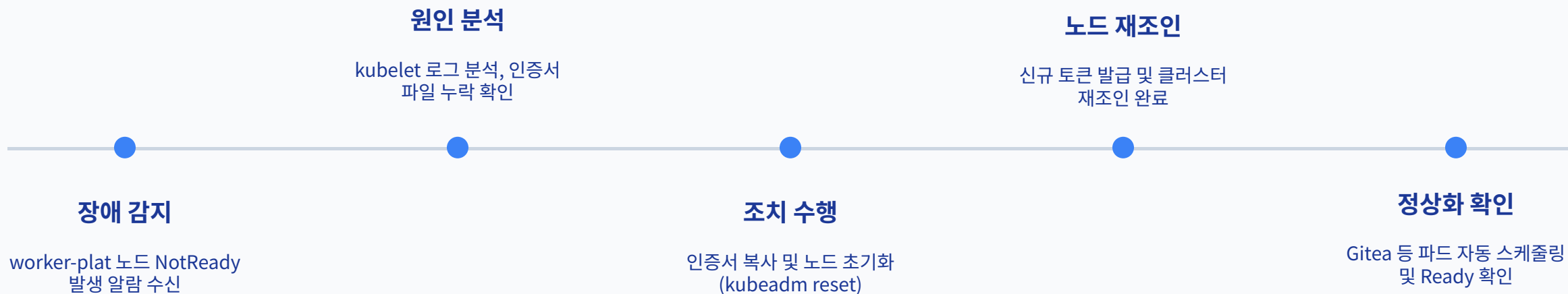
 **임계치 초과 실시간 알람 발동 (Firing)** 쿠버네티스 파드 비정상 다운 및 데이터소스 연결 에러(`DataSourceError`) 감지 시 Prometheus Alert Rule을 통해 능동적 발람 모드로 즉각 전환

 **Gitea 연동형 자동 장애 티켓 퍼블리싱** 단순 알람 발송을 넘어 웹훅(Webhook) 트리거 메커니즘을 적용, 협업 플랫폼인 **Gitea 이슈 보드에 장애 티켓 자동 생성 및 담당자 할당** 연동 완료

 **장애 극복 이력의 자산화 (Runbook)** 티켓 수명 주기 관리를 통해 장애 유발 노드 및 처리자 트래킹 히스토리를 데이터화하여 재발 방지책 수립 기반 마련



| 실전 장애 대응: 노드 복구 자동화



관제 인프라 구축의 정량적 성과 및 기술적 차별점

비교 지표 항목	기존 인프라 운영 방식 (As-Is)	본 프로젝트 구축 아키텍처 (To-Be)
장애 인지 방식	사용자 불편 민원 접수 및 인프라 사후 인지	Alertmanager 실시간 룰 자동 감지 (Firing)
로그 트래킹 메커니즘	개별 노드 원격 터미널 접속 후 분산 파일 수동 grep	Loki 중앙 레이블링 브라우저를 통한 통합 초고속 쿼리
평균 장애 대응 시간 (MTTR)	원인 파악 및 이력 분석에 평균 30분 이상 소요	Gitea 티켓 자동 발행 연동으로 실시간 트래킹 (5분 이내)
스토리지 효율성	전체 원시 텍스트 백업 저장으로 오버헤드 유발	인덱스 최소화 시계열 압축 저장 정책 기 가동

가시성 아키텍처 향후 고도화 로드맵



자동 자가 치유 (Auto-Healing)

Alertmanager가 감지한 임계치 알람을 Ansible 스크립트 및 웹훅 리시버와 연계하여, 수동 개입 없는 컨테이너 재부팅과 좀비 프로세스 자동 격리 파이프라인 구축



분산 트레이싱 (Tempo 스택 확장)

기존 메트릭(Prometheus) 및 로그(Loki)에 Grafana Tempo를 추가하여 MSA 분산 트랜잭션의 병목 구간을 시각화하고, 통합 가시성(Observability) 모델 구축

보안

프로젝트 개요 및 배경



다양한 인프라 구조

VM, 컨테이너, 물리 서버 혼재로 일관된 보안 정책 적용이 어려운 상황



보안 사각지대 존재

방화벽·Kubelet 등 에이전트 설치 불가 장비의 로그가 미수집 상태



수동 대응 한계

공격 탐지 후 관리자 개입까지 골든타임 내 대응 실패 위험

▼ 해결 방향

Wazuh SIEM + Prometheus/Loki 기반 통합 관제 플랫폼 구축으로 탐지·대응·예방을 자동화

>> 단일 대시보드에서 보안 위협과 인프라 메트릭을 동시 모니터링

Wazuh 보안 정책 구성 및 일괄 배포 자동화



자동화된 보안 정책 배포

서버 10대에 대한 개별 설정의 번거로움과 정책 누락을 방지하기 위해 SIEM 서버 기반 쉘 스크립트 일괄 배포 체계를 구축

구분	서버명	IP 주소	역할
SIEM	siem-01	172.16.30.85	Wazuh Manager / SIEM
HAProxy	haproxy-01	172.16.20.26	Load Balancer
HAProxy	haproxy-02	172.16.20.27	Load Balancer
Monitoring	monitor-01	172.16.30.90	Monitoring Server
Kubernetes Master	k8s-master-01	172.16.30.30	Control Plane
Kubernetes Master	k8s-master-02	172.16.30.32	Control Plane
Kubernetes Master	k8s-master-03	172.16.30.33	Control Plane
Kubernetes Worker	k8s-worker-01	172.16.30.45	Worker Node
Kubernetes Worker	k8s-worker-02	172.16.30.46	Worker Node
Kubernetes Worker	k8s-worker-03	172.16.30.47	Worker Node
Kubernetes Worker	k8s-worker-plat	172.16.30.40	Platform Worker Node



5단계 통합 구성 프로세스

에이전트 등록부터 pfSense 연동, 로그 수집, 인덱서 저장, Active Response 설정까지 단일 스크립트로 자동 실행되는 구조 설계

```
kosa@siem-01:~/wazuh$ tree
.
├── 00-ssh_key_deploy.sh
├── 01-agent_log_collect.sh
├── 02-manager_json_process.sh
├── 03-pfsense_syslog.sh
├── 04-indexer_store.sh
├── 05-active_response.sh
├── 06-wazuh_all.sh
├── config
│   ├── certs
│   └── config.yml
└──
```

3 directories, 8 files
kosa@siem-01:~/wazuh\$ |

보안 이벤트 데이터 흐름 및 통합 가시성



전방위 로그 수집

- SSH 인증, 시스템 로그, FIM, K8s API 실시간 수집
- 1514/TCP 포트를 통한 실시간 전송 및 분석



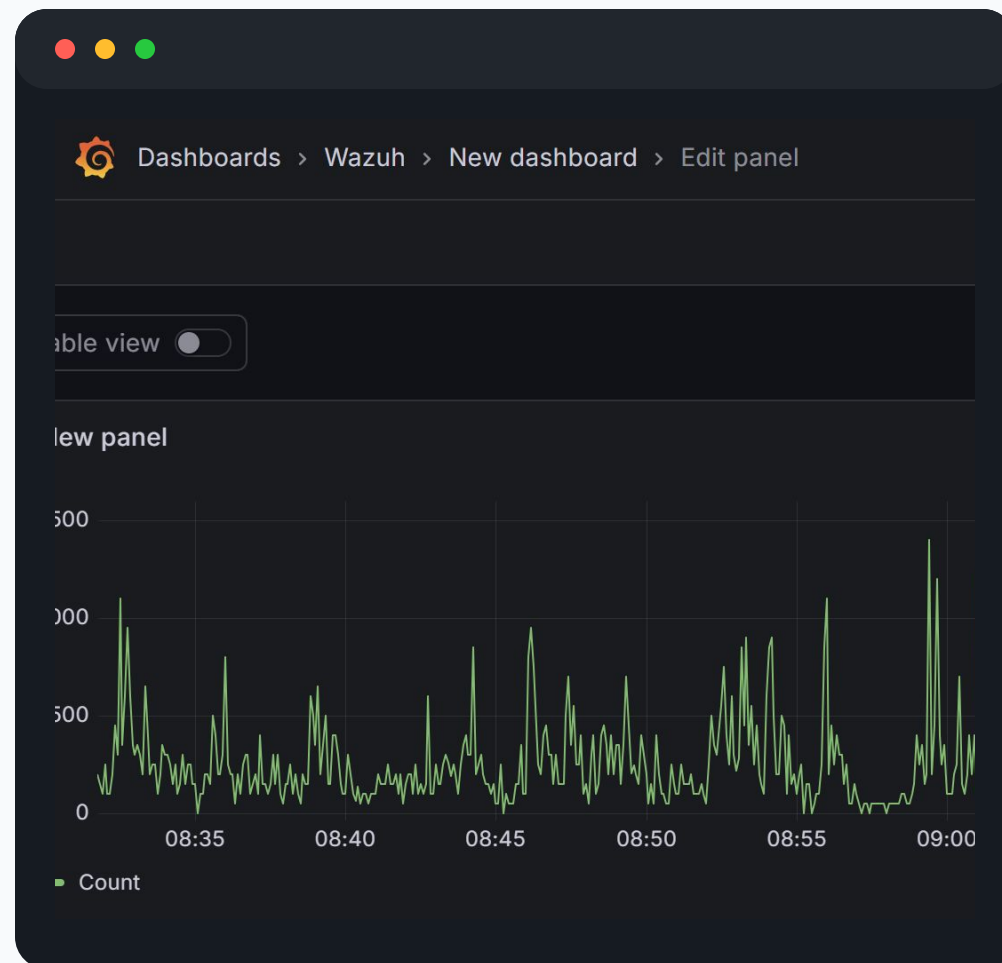
Filebeat 기반 데이터 정제

- JSON 경고문을 Filebeat가 읽어 Indexer에 저장
- 대용량 로그 검색 성능 및 데이터 정제성 확보



통합 관제 싱글 뷰 (SSoT)

- Wazuh Dashboard와 Grafana의 심리스한 연동
- 인프라 메트릭과 보안 위협의 단일 화면 관제



방화벽 안정성을 위한 Agentless(pfSense Syslog)

방화벽 가용성 최우선 확보

- pfSense 에이전트 미설치로 네트워크 장애 방지
- 514/UDP Syslog 방식으로 장비 부하 최소화
- 실시간 로그 전달로 가용성 및 성능 동시 확보

```
echo ""
echo "[3/3] pfSense Syslog 수신 확인 중 (5초)..."
echo "    pfSense에서 로그가 발생해야 패킷이 잡힙니다."
echo ""

timeout 5 tcpdump -i any -n udp port 514 2>/dev/null | head -10 || true

echo ""
echo "    [수동 확인 명령어]"
echo "    sudo tcpdump -i any udp port 514"
```

계층적 방어 체계 (Defense in Depth)

- 외부 방화벽 돌파 시에도 내부 에이전트의 개별 탐지
- pfSense와 Agent 다중 탐지 레이어로 보안 강화
- 보안 시스템의 단일 장애점(SPOF) 완벽 제거

```
kosa@siem-01:~/wazuh$ sudo grep -A5 "<connection>syslog</connection>" /var/ossec/etc/ossec.conf
<connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>172.16.0.0/16</allowed-ips>
  <local_ip>172.16.30.85</local_ip>
</remote>
kosa@siem-01:~/wazuh$ |
```

과정



Single Source of Truth — Wazuh Dashboard + Grafana 연동으로 보안과 인프라를 한 화면 통합 관제

 Wazuh Dashboard → Grafana DataSource 연동 → 통합 뷰 확인

포트 구성: Agent → Manager: 1514/TCP | pfSense → Manager: 514/UDP |
Filebeat → Indexer: 9200/TCP

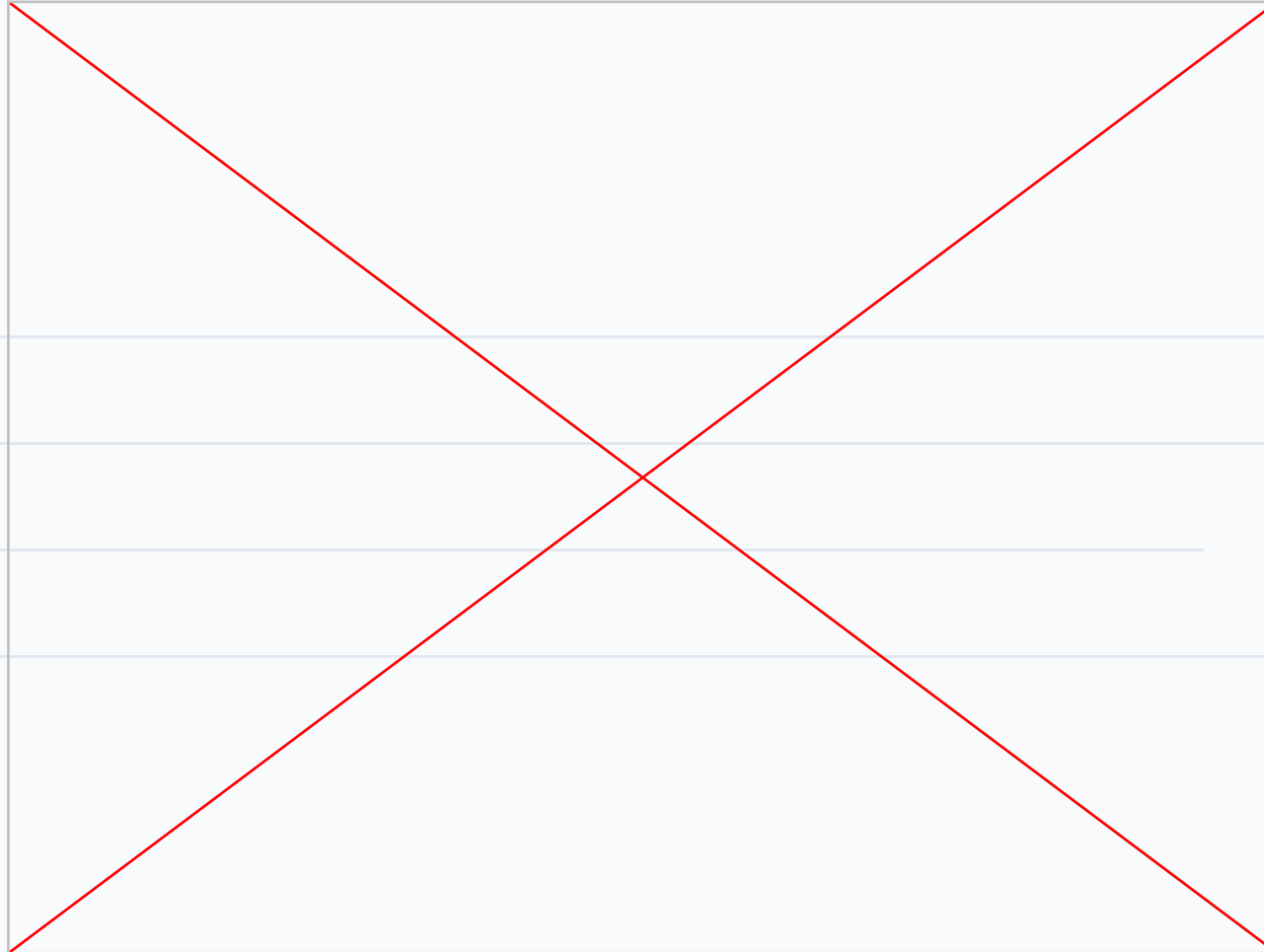
위협 자동 차단: Active Response 정책

보안 위협 시나리오에 따른 세부 대응 규칙 및 자동화 정책을 아래와 같이 구성하였습니다.

Rule ID	탐지 조건 (Detection)	차단 범위 (Scope)	해제 시간 (Timeout)
5720	SSH 무차별 대입 (Brute Force)	해당 서버 로컬 차단	10분
5763	SSH 인증 반복 실패	해당 서버 로컬 차단	10분
100201	pfSense 포트스캔 탐지	전체 서버 10대 차단	30분
100211	OpenVPN 무차별 대입 공격	전체 서버 10대 차단	30분

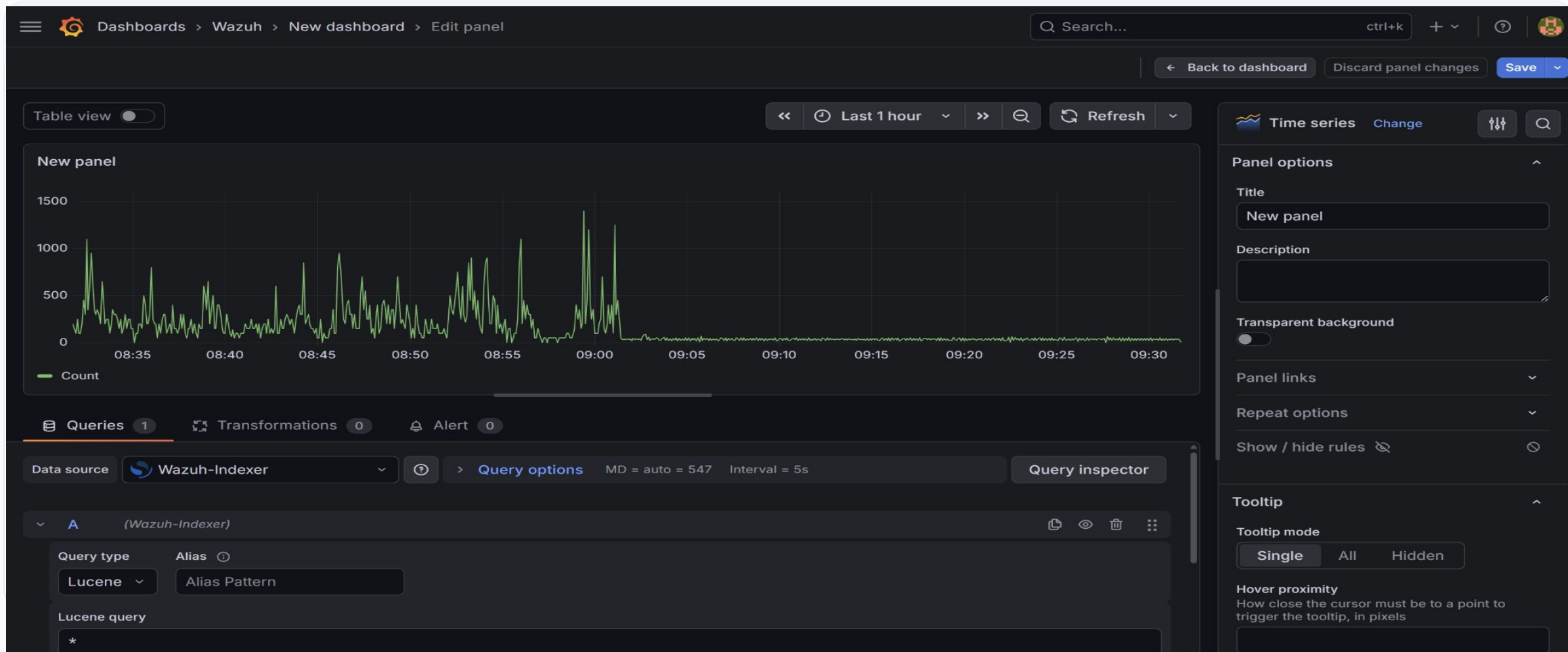
웹 스크립트 시연 영상

<https://drive.google.com/file/d/1RXrGtysNs6EmK663L8kCxX77afHmNiju/view?resourcekey>



와주 대시보드와 그라파나 대시보드 연동

브루트포스(Brute-force) 공격으로 인한 로그인 실패 패턴을 실시간 탐지하여 이벤트 폭증 구간을 시각화하였습니다.



docker 보안 가이드 진단 현황

DO-01

양호

호스트 OS 주요 자원 접근 제어

DO-04

취약

컨테이너 권한 제어 (root 실행 방지)

DO-02

취약

SSL/TLS 적용 (Docker 데몬 원격 접속)

DO-03

양호

인증-권한 제어 (Docker 소켓 보호)

docker 보안 가이드 취약 항목 대응1

DO-02 SSL/TLS 미적용 (Docker 데몬 원격 접속)

보안효과 데몬-클라이언트 통신 암호화로 중간자 공격 및 비인가 원격 접근 방지

대응방법

/etc/docker/daemon.json
"tlsverify": true | "tlscacert": "/etc/docker/ca.pem
"tlscert": / "tlskey" 경로 지정

DO-04 컨테이너 root 실행 / 권한 미제한

보안효과 권한 상승 공격 및 컨테이너 탈출 발생 시 호스트 피해 범위 최소화

대응방법

--user=1000:1000 | --security-opt=no-new-privileges
--privileged 사용 금지 | Dockerfile: USER 10001

kubernetes 보안 가이드 진단 현황

KU-01 취약 API Server 익명 접근 차단 / SA 토큰 검증	KU-08 양호 Controller Manager 서비스 계정 자격증명	KU-09 취약 Controller Manager TLS 적용
KU-02 양호 RBAC 권한 제어 (Node, RBAC)	KU-10 취약 특권 컨테이너 실행 차단	KU-11 양호 설정 파일 권한 관리 (kube-apiserver)
KU-03 양호 API Server SSL/TLS 적용	KU-12 양호 설정 파일 권한 관리 (scheduler)	KU-14 취약 Kubelet 커널 파라미터 보호
KU-04 취약 Admission Control Plugin	KU-05 취약 감사 로그 (Audit Log) 설정	KU-06 양호 etcd SSL/TLS 통신
KU-07 취약 etcd 저장 데이터 암호화		

kubernetes 보안 가이드 주요 취약 대응1

KU-01 API Server 익명 접근 차단 / SA 토큰 검증 누락

보안효과 인증 없는 API 접근 차단 + 삭제된 서비스 계정 토큰 우회 방지

대응방법 `--anonymous-auth=false` | `--service-account-lookup=true` | `--basic-auth-file`, `--token-auth-file` 사용 금지

KU-04 Admission Control Plugin 미설정

보안효과 권한 없는 리소스 생성 및 보안 정책 우회 방지, 일관된 보안 정책 강제

대응방법 `--enable-admission-plugins=NodeRestriction,PodSecurityPolicy,ServiceAccount,NamespaceLifecycle`

KU-05 감사 로그 미설정

보안효과 API Server 접근/변경 이력 추적으로 이상 행위 탐지 및 사고 원인 분석

대응방법 `--audit-log-path=/var/log/kubernetes/audit.log` | `--audit-policy-file=...` | `--audit-log-maxage=30` | `--audit-log-maxbackup=10` | `--audit-log-maxsize=100`

kubernetes 보안 가이드 주요 취약 대응2

KU-07 etcd 저장 데이터 미암호화

보안효과 etcd 직접 접근 시에도 Secret 등 민감 데이터의 평문 노출 방지

대응방법 `--encryption-provider-config=/etc/kubernetes/encryption-config.yaml`

KU-10 특권 컨테이너 실행 허용

보안효과 호스트 커널 무제한 접근 차단 → 컨테이너 탈취 및 시스템 침해 위협 대응

대응방법 `--allow-privileged=false | securityContext.privileged=false | allowPrivilegeEscalation=false`

KU-14 Kubelet 커널 파라미터 보호 미설정

보안효과 컨테이너 내부에서 호스트 커널 파라미터 변경 차단으로 보안 설정 우회 방지

대응방법 `/var/lib/kubelet/config.yaml | protectKernelDefaults: true`

사전 예방을 위한 인프라 보안 가이드 수립



Linux Host OS

Root 로그인 제한 및 비밀번호 복잡성, 불필요한 서비스 포트 제거 기준 수립



Docker Runtime

비루트 권한 실행(Rootless), 이미지 서명 검증 및 리소스 제한 설정 가이드



Kubernetes

RBAC 권한 최소화, API 감사 로그 활성화 및 네트워크 정책 적용

"실시간 탐지(Detection), 자동 차단(Response), 그리고 보안 가이드를 통한 예방(Prevention)의 세 가지 계층이 상호 보완되는 보안 체계를 완성했습니다."

한계 및 개선

한계 및 향후 개선

CRITICAL

인프라 확장 자동화 도입 지연 (Karpenter 미적용)

프로젝트 기간 제한으로 인해 **Karpenter** 등 AWS 고도화 오토스케일러를 도입하지 못해, 초기 클러스터 자원 최적화 및 유연한 노드 확장에 한계가 있었습니다.

트래픽 분산 제어의 한계 (Failover 방식의 국한)

현재 온프레미스 장애 시에만 AWS로 전환되는 구조이나, 서비스 안정성을 위해 특정 비율(Weight-baseded)로 트래픽을 상시 분산하는 구성이 더 효율적이었을 것으로 판단됩니다.

클라우드 데이터베이스(RDS) 도입 부재

고가용성 및 자동 백업 체계를 갖춘 클라우드 전용 DB(RDS)를 활용하지 못해, 온프레미스 DB 관리 부하 및 장애 복구 탄력성에 한계가 있었습니다.

Q & A

A short, horizontal blue line positioned directly beneath the 'Q & A' text.